UNIFIED DIAGNOSTIC LABORATORY

+ BLOOD BANK + BILLING SAAS

Business Requirements | Product Requirements | System Design | Database Design | Implementation Plan

Product principle

One login. One application. One patient record. Pathology, billing, reporting and blood bank are modules inside the same workspace. Many users and many computers can work simultaneously on the same tenant data.

Item	Detail
Document version	1.0
Prepared for	Unified Lab Pathology + Blood Bank SaaS product
Date	08 October 2026
Primary deployment	Cloud, offline-first desktop, on-prem/local server, hybrid
Target market	Diagnostic labs, pathology centres, blood banks, hospitals with in-house labs, collection centres
Primary client surfaces	Web, Windows desktop, mobile apps, analyzer connector
Architecture goal	Lightweight, modular, multi-tenant, role-based, auditable, offline-capable
Compliance posture	Designed to support accreditation and regulatory controls; final compliance requires validation, SOPs and implementation against applicable standards/guidelines.

1. Executive Summary

This document defines the requirements and implementation blueprint for a unified diagnostic laboratory platform combining pathology laboratory operations, patient registration, billing, referral management, report generation, blood bank management, inventory, analyzer integration, role-based access control, subscriptions, licensing and offline-first operation.

The product is deliberately designed as one application rather than separate pathology, billing and blood-bank products. A diagnostic centre can enable all modules or only the modules it needs. Reception desks can use the same system from multiple computers, while technicians, pathologists, accountants and blood-bank staff can work simultaneously on the same shared data.

The platform must support cloud SaaS, Windows desktop, mobile and local/offline deployments. In offline mode, the installation continues operating against local data and synchronizes with the cloud when connectivity returns. Annual offline licensing is supported; access should transition to a controlled renewal state after expiry without silently deleting or corrupting clinical data.

2. Product Vision

Build a clean, fast and configurable Laboratory Information and Operations Platform that is simple enough for a reception user but deep enough for a diagnostic centre with multiple departments, branches, analyzers and blood-bank workflows.

Design principle	Requirement
Unified workspace	One login, one application shell, common navigation and shared patient identity across pathology, billing and blood bank.
Configurable	Modules, screens, fields, permissions, report templates, pricing, discounts and referral rules can be configured per tenant/branch.
Concurrent	Multiple staff members and computers can perform different or similar tasks at the same time with transaction-safe writes and live queue updates.
Offline-first	Core operations continue without internet in supported offline deployments. Sync is explicit, encrypted and conflict-aware.
Auditable	Clinical result changes, report amendments, approvals, billing overrides, stock actions and administrative changes have audit records.
Lightweight	Fast initial load, lazy-loaded modules, compact APIs, background jobs and optimized queries.
Integration-ready	Analyzer connectors, HL7/FHIR adapters, messaging, storage and payment integrations use adapter interfaces rather than hard-coded vendor logic.

3. Scope and Product Boundaries

3.1 In scope

- Multi-tenant SaaS and single-organization deployments.
- One unified app shell with modules for pathology, billing, blood bank, referrals, reports, inventory, users, RBAC and administration.
- Multi-user and multi-PC simultaneous operation.
- Patient registration, search, order management, sample lifecycle, result management and report delivery.
- Blood donor and blood component management, inventory, testing, cross-matching and issue/return/discard workflows.
- Billing, GST configuration, discounts, payments, refunds, credits, pricing and referral commissions.
- 100+ starter report templates plus a reusable custom report designer.
- Analyzer integrations through configurable connectors and mapping rules.
- Web application, Windows desktop application, mobile apps and analyzer connector/service.
- Cloud storage, local storage, scheduled backup and import/export.
- Annual SaaS/offline licensing and feature/module entitlements.
- RBAC with granular actions, role assignment, branch and department scope.

3.2 Out of scope for the initial platform

- Full hospital EHR/HMIS replacement including nursing, OT, radiology PACS and pharmacy unless later added as separate modules.
- Device-specific analyzer drivers that are not technically documented by the analyzer vendor. These should be implemented as connector adapters after protocol verification.
- Guaranteeing accreditation or regulatory certification by software alone. The software should provide controls and evidence; the operating organization remains responsible for SOPs, staff competence, validation and accreditation requirements.
- Automated medical diagnosis. The application may calculate, flag or format results, but clinical interpretation remains under authorized personnel.

4. Users, Personas and Work Environments

Persona	Primary tasks	Typical device
Organization Admin	Tenant settings, users, roles, branches, pricing, modules, reports, license	Web/Desktop
Receptionist	Patient registration, order, billing, payment, barcode, print report	Desktop/Web
Billing Executive	Invoices, discounts, payments, refunds, ledger, daily closing	Desktop/Web
Lab Technician	Sample receive/collect/process, result entry, QC-related tasks	Desktop/Web/Mobile

Persona	Primary tasks	Typical device
Pathologist	Result review, verification, approval, digital signature, amendments	Desktop/Web/Tablet
Blood Bank Technician	Donor, screening, collection, component processing, stock and issue	Desktop/Web/Mobile
Blood Bank Manager	Approval, inventory controls, expiry, wastage and regulatory reports	Web/Desktop
Referral Manager	Doctor/hospital/lab relationships and commission settlement	Web/Desktop
Accountant	Ledger, collections, refunds, reconciliation and reports	Web/Desktop
Collection Centre User	Patient registration, sample collection, dispatch and receipt tracking	Web/Mobile
Super Admin	SaaS plans, tenants, support, feature flags, templates, licenses and system health	Web

Key operating model

A pathology centre may have four reception computers and four different reception users. Each user can have the same Receptionist role and work simultaneously on registration, billing, report printing and sample workflows. The permission model controls actions; the computer does not define the role.

5. Functional Architecture

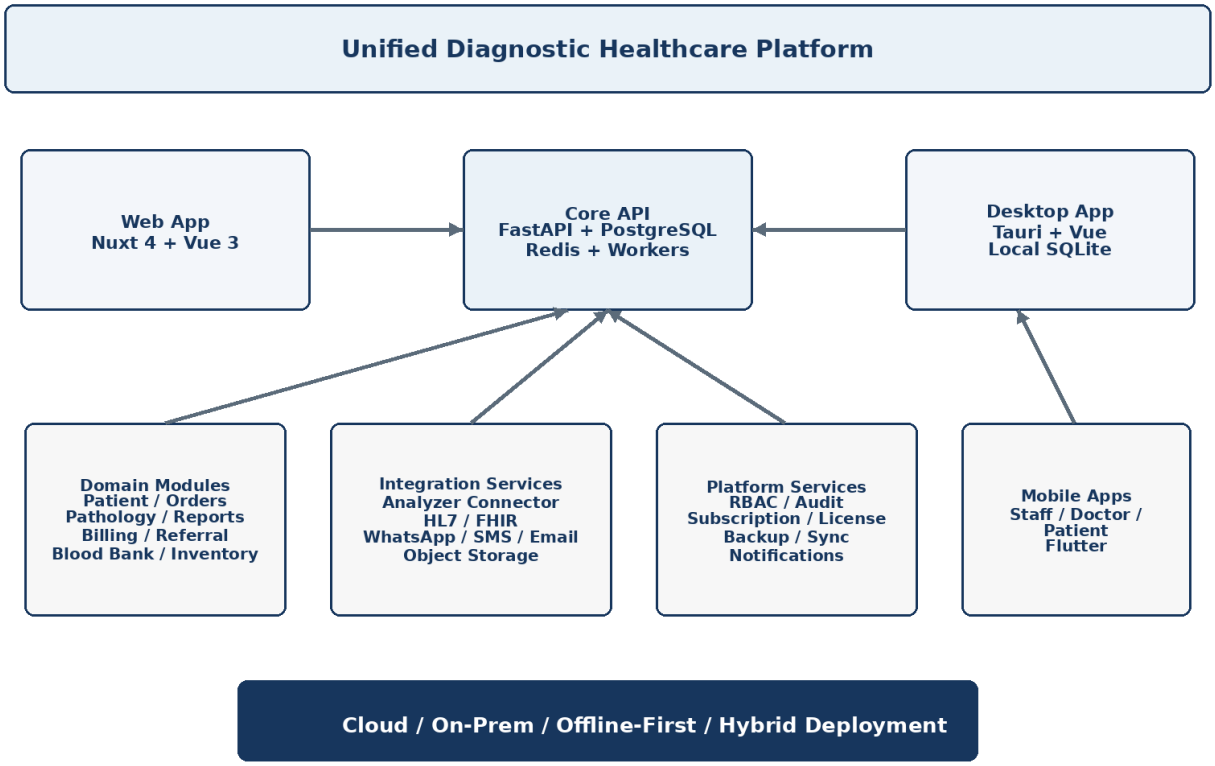


Figure 1. Unified application architecture

5.1 Core application modules

#	Module	Primary scope
1	Platform Administration	Tenant, branches, modules, settings, branding, subscription, licenses, feature flags, audit and backup.
2	Identity and RBAC	Authentication, sessions, roles, permissions, branch scope, department scope, data scope, overrides.
3	Patient Management	Registration, UHID, demographics, addresses, identifiers, history, documents, consent and duplicate detection.
4	Pathology / LIMS	Test master, profiles, packages, orders, samples, barcodes, worklists, results, validation, approval and report release.
5	Billing	Rate lists, invoices, discounts, GST configuration, payments, refunds, credits, receipts, cash closure and reconciliation.

#	Module	Primary scope
6	Referral Management	Doctors, hospitals, external labs, referral contracts, discount rules, commission rules, ledger and settlement.
7	Reporting	Report designer, template library, versioning, QR verification, signatures, delivery and report history.
8	Blood Bank	Donors, screening, donation, components, testing, quarantine/release, stock, cross-match, issue, return, discard and adverse events.
9	Inventory	Reagents, consumables, blood components, lot/batch, expiry, min/max stock, purchase/issue/adjustment and alerts.
10	Analyzer Integration	Connector service, ASTM/HL7/TCP/serial/file input, mapping, raw messages, result import, error queue.
11	Notifications	Email, SMS, WhatsApp, push, templates, queues, delivery tracking and retry.
12	Analytics	Operational dashboards, TAT, revenue, test volumes, stock, referral, user activity and quality indicators.
13	Import/Export	CSV/XLSX/JSON/PDF exports, bulk imports, mapping, validation, dry run and rollback where feasible.
14	Sync and Offline	Local database, change queue, sync engine, device registration, conflict handling and status dashboards.

6. Unified UI/UX Requirements

The application should look and behave as one product. Users should not feel that pathology, billing and blood bank are separate systems. The left navigation, header, global search, patient drawer, notification center, shortcuts and action patterns should remain consistent across modules.

6.1 Recommended global navigation

- Dashboard
- Patients
- Orders & Samples

Pathology
Reports
Billing
Referral Partners
Blood Bank
Inventory
Doctors / Hospitals / Labs
Templates
Analytics
Imports & Exports
Administration

Top bar: Global Search | Notifications | Branch | User Profile

6.2 Configurable home dashboard

- Widgets are permission-aware and module-aware. If Blood Bank is disabled, blood-bank widgets are not shown.
- Admin can configure default widgets per role.
- Users can pin frequent actions: New Patient, New Order, Collect Sample, Quick Billing, Print Report, Blood Issue.
- Operational queues should be visible without navigating through multiple screens.
- Global patient search should return patient, UHID, mobile, order, invoice, report and sample references where permitted.

7. RBAC, Roles and Permission Model

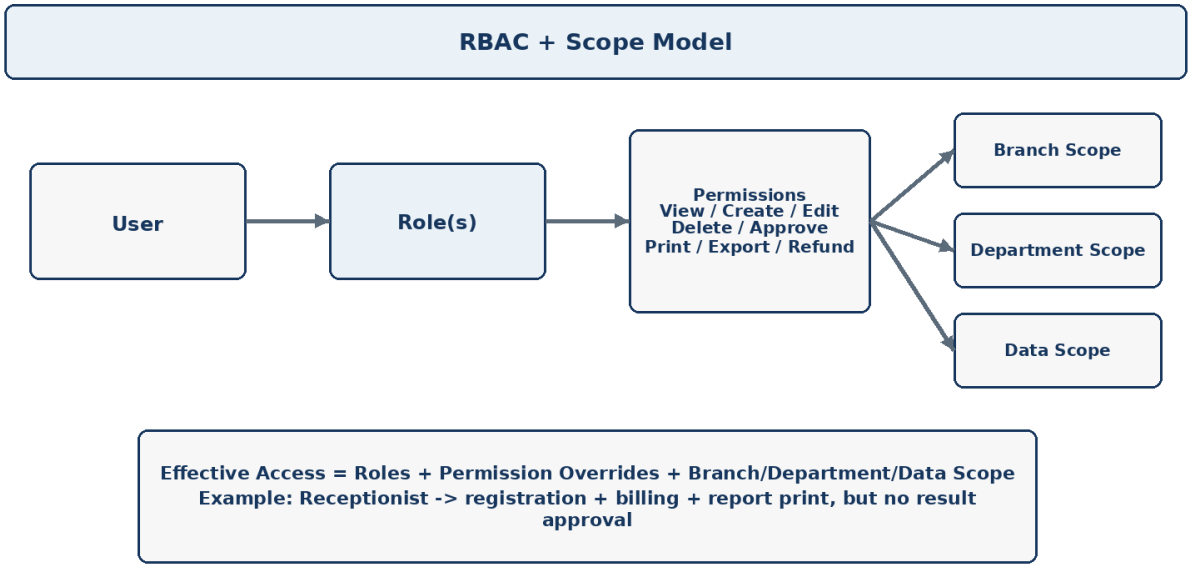


Figure 2. Role, permission and scope model

Permissions must be granular. A single manage toggle is insufficient for a healthcare application because viewing, editing, approving, refunding, printing and exporting can have different risk levels.

7.1 Permission naming standard

<module>.<resource>.<action>

Examples:
patient.patient.create
patient.patient.view
billing.invoice.create
billing.invoice.discount
billing.invoice.refund
lab.result.enter
lab.result.verify
lab.result.approve
report.report.amend
blood.stock.issue
blood.stock.discard
admin.user.manage

7.2 Role structure

Role	Typical access
Super Admin	SaaS-wide administration; no tenant clinical access unless explicitly granted for support.
Organization Admin	Tenant-wide configuration, users, roles, modules, pricing, templates and reports.
Lab Admin	Pathology and laboratory configuration and operational oversight.
Receptionist	Patient registration, order, billing, payment, barcode, report view/print.
Billing Executive	Invoices, payments, discounts within assigned limits, refunds if permitted.
Lab Technician	Samples, worklists, result entry and technical operations.
Pathologist	Result review, report approval, signature, amendment and critical-result workflow.
Blood Bank Technician	Donor, donation, component, test, stock and cross-match operations.
Blood Bank Manager	Approval, stock controls, wastage, regulatory reporting and oversight.
Accountant	Financial reports, settlement, reconciliation, ledger and audit views.
Custom Role	Any configurable combination of permissions, branch scope and department scope.

7.3 Multiple roles per user

A user may have more than one role. The effective permission set is the union of assigned role permissions, subject to explicit deny/override policy. The system must store why an elevated permission exists and log sensitive actions.

7.4 Scope controls

Scope	Examples
Organization	All branches in one tenant
Branch	Noida branch only
Department	Hematology + Biochemistry
Resource owner	Records created by the user, where required
Data class	Clinical, billing, blood-bank or administrative
Action limit	Discount up to 10%; refund up to configured amount

8. Business Requirements

ID	Area	Requirement
BR-001	Unified operations	Pathology, billing and blood bank must be available from one login and application shell.
BR-002	Multi-user	Multiple users can work simultaneously on the same tenant and branch.
BR-003	Multi-PC	Multiple computers in reception, lab and blood bank can operate concurrently against shared data.
BR-004	Configurable modules	Tenant admin can enable/disable modules and selected features.
BR-005	Annual licensing	Offline installations can be licensed for a fixed period and renewed annually.
BR-006	Cloud subscription	Cloud deployments support subscription plans, entitlements and usage limits.
BR-007	Offline continuity	Supported offline deployments continue core operations when internet is unavailable.

ID	Area	Requirement
BR-008	Sync	Changes made offline are queued and synchronized with the cloud when connectivity returns.
BR-009	Patient identity	One patient identity is shared across orders, billing, reports and blood-bank records.
BR-010	Configurable pricing	Rates, packages, corporate rates, referral rates and discounts are configurable.
BR-011	Referral accounting	Doctor/hospital/lab referral commission and patient discount rules are calculated automatically.
BR-012	Report customization	Users with permission can choose and configure report templates; final approval is controlled.
BR-013	Analyzer import	Analyzer results can be imported automatically where protocol support is available.
BR-014	Audit	Sensitive changes have immutable or tamper-evident audit records.
BR-015	Import/export	Data can be imported/exported with validation, logs and permissions.
BR-016	Backup	Tenant and installation backup/restore workflows are available.
BR-017	Notifications	Report-ready and operational alerts can be delivered through configured channels.
BR-018	Integration	API and interoperability adapters support external systems without changing the core domain model.

9. End-to-End Pathology Workflow

Patient Search / New Patient
-> Registration / Duplicate Check
-> Referral Source
-> Test / Profile / Package Selection
-> Pricing + Discount Rule
-> Invoice + Payment
-> Sample Collection
-> Barcode Print
-> Sample Receive / Accession

```
-> Analyzer / Manual Result
-> Result Validation
-> Abnormal / Critical Flag
-> Technical Verification
-> Pathologist Review / Approval
-> Report Render + Signature + QR
-> Print / Download / WhatsApp / SMS / Email
-> Amendment workflow if required
```

9.1 Patient registration requirements

- Patient number/UHID generation with configurable prefix and branch scope.
- Duplicate detection using configurable matching rules such as mobile, name + DOB, external identifier and other tenant-defined criteria.
- Ability to register a patient without forcing every optional field.
- ABHA field/linking should be treated as an integration/identity feature and must follow the current ABDM implementation and consent requirements at the time of production deployment.
- Patient merge must be restricted, audited and reversible through a controlled workflow where feasible.

9.2 Sample lifecycle

```
ORDERED -> COLLECTED -> BARCODED -> RECEIVED -> ACCEPTED
                                  |
                                  -> REJECTED (reason required)

RECEIVED -> IN PROCESS -> RESULT ENTERED -> VERIFIED -> RELEASED
```

10. Test Master and Result Engine

Tests are not just display names. Each test must be a structured definition that can drive ordering, billing, specimen handling, analyzer mapping, result entry, reference ranges and reporting.

Field	Purpose
test_code	Unique stable internal code
display_name	Human-readable test name
department_id	Hematology, Biochemistry, Serology, etc.
specimen_type	Blood, serum, plasma, urine, swab, etc.
container_type	Tube/container definition
method	Method/assay
unit	Result unit
decimal_precision	Display precision
reference_range	Age/sex/condition-specific range
critical_low / high	Critical thresholds

Field	Purpose
calculation_formula	Derived value calculation
analyzer_code	Machine mapping identifier
loinc_code / local_code	Interoperability or local coding field
turnaround_time	Operational TAT
active_from / active_to	Versioned validity
report_group	Placement in report template

10.1 Profiles and packages

- Test profile groups multiple analytes for ordering and reporting.
- Package can bundle tests with commercial pricing, optional discount and referral rules.
- An individual test may appear in multiple profiles/packages without duplicating the underlying test definition.
- Package price should not destroy line-level auditability; invoice lines should preserve the final allocated amounts.

11. Blood Bank Module

The Blood Bank module shares patient, user, audit, billing, inventory and reporting infrastructure with the rest of the platform but maintains its own domain state machine and stricter authorization rules.

11.1 Donor workflow

```
Donor Registration
  -> Identity / History
  -> Screening Questionnaire
  -> Eligibility Decision
  -> Donation
  -> Component Processing
  -> TTI / Required Testing
  -> Quarantine
  -> Release / Reject / Discard
  -> Inventory
```

11.2 Inventory states

```
COLLECTED -> TESTING -> QUARANTINED -> RELEASED -> RESERVED -> ISSUED
                                  |
                                  -> EXPIRED / DISCARDED
ISSUED -> RETURNED (only under approved rules)
```

Capability	Requirements
Donor	Registration, history, questionnaire, deferral, repeat donor tracking
Donation	Donation event, collection details, component generation
Testing	Blood group, TTI/infectious screening fields, quality checks, verification
Components	RBC, plasma, platelets, cryoprecipitate and configurable components
Barcode	Configurable labels; support for required local or ISBT-style encoding where implemented and validated
Stock	Real-time availability, location, blood group, component, lot, expiry
Cross-match	Compatibility workflow, patient/sample linkage, test results and approval
Issue	Request, reservation, issue, return, discard and traceability
Alerts	Low stock, expiring units, pending testing, quarantine and exceptions
Reporting	Regulatory and operational reports with configurable formats

External blood-bank ecosystem note

The e-RaktKosh platform describes donor management, blood grouping, TTI screening, component preparation, centralized inventory, alerts and statutory reporting. The product should therefore model these as first-class domain concepts and keep an integration boundary for current government workflows rather than assuming a static API contract.

12. Billing, Discount and Referral Commission Engine

Billing must work from the same patient/order context as pathology. A receptionist should be able to register a patient, select tests, choose a referral source, apply an eligible discount, collect payment and print the receipt without moving to another application.

12.1 Pricing resolution order

Base Test Price
-> Branch Rate / Tenant Rate
-> Package / Profile Rule
-> Corporate / Contract Rate (if applicable)
-> Referral Discount Rule

-> Manual Discount (if permitted)
-> Taxes / GST configuration
-> Final Invoice Amount

12.2 Referral rule model

Rule field	Example
partner_type	Doctor / Hospital / Lab / Collection Centre / Corporate
partner_id	Dr. X
scope	All tests / Department / Test / Package
effective_from	2026-10-01
effective_to	Optional
patient_discount_pct	5%
commission_pct	10%
commission_fixed	Optional
priority	Higher priority wins if multiple rules match
approval_required	Yes/No
status	Active/Inactive

12.3 Financial rules

- Invoice totals must be deterministic and reproducible from stored line/rule snapshots.
- Discount and commission rules used at invoice time must be preserved with the invoice; later rule changes must not rewrite history.
- Refunds and cancellations must generate accounting/audit events rather than deleting invoices.
- Commission settlement should use a ledger model, not just a summary field on the doctor record.
- Authorization thresholds should support amount-based approval for discounts and refunds.

13. Report Engine and Custom Report Designer

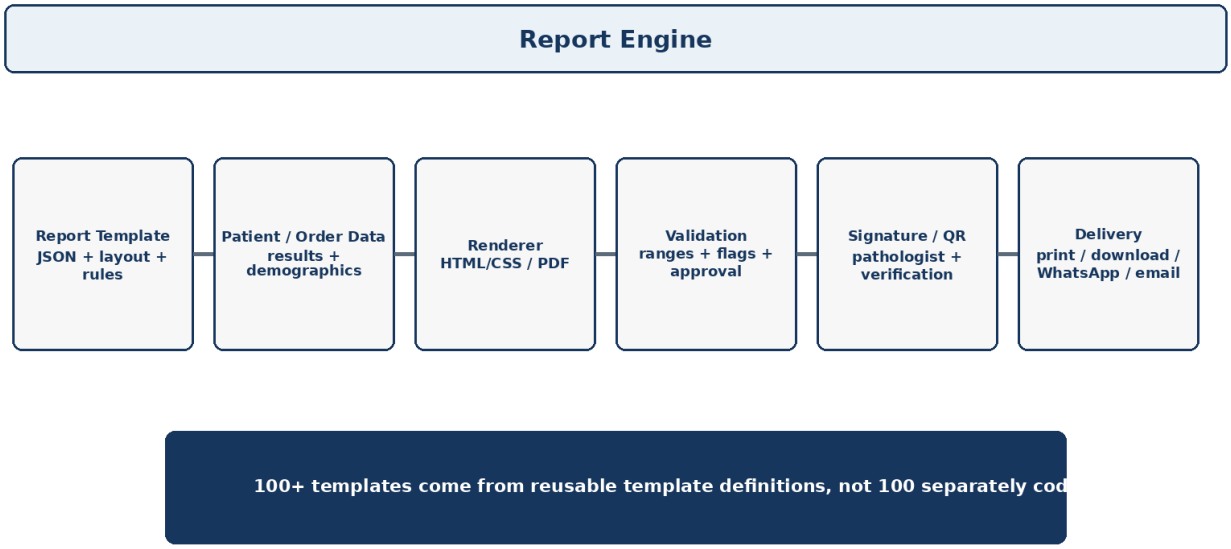


Figure 3. Report generation pipeline

The system should ship with 100+ starter templates, but these templates should be definitions stored in the database/object store rather than 100 hard-coded screens. The same renderer should support preview, PDF generation and print output.

13.1 Report designer components

- Header, footer, logo and organization branding.
- Patient demographics block.
- Doctor/referral source block.
- Test/result table with reference range and flags.
- Grouped sections and profiles.
- Conditional visibility rules.
- Charts, histograms and image blocks where clinically appropriate.
- Barcode and QR verification blocks.
- Digital signature and authorization block.
- Page numbering, print margins and paper size.
- Watermark and status such as DRAFT/FINAL/CORRECTED.
- Multi-language labels and custom static text.
- Template versioning with publish/rollback capability.

13.2 Sample template model

```
{
  "template_id": "CBC-A4-001",
  "version": 3,
  "page": {"size": "A4", "orientation": "portrait"},
  "blocks": [
    {"type": "patient", "x": 24, "y": 38, "w": 540, "h": 62},
```

{"type": "result_table", "source": "CBC", "x": 24, "y": 120, "w": 540, "h": 360},
{"type": "qr", "source": "report_verification", "x": 480, "y": 700}
],
"footer": {"show_page_number": true}
}

14. Analyzer Integration Architecture

```
Analyzer
  |
  +-- Serial / TCP / ASTM / HL7 / File Drop
  |
  v
Analyzer Connector Service
  |-- Device Health
  |-- Raw Message Capture
  |-- Parser
  |-- Mapping Engine
  |-- Patient/Sample Matching
  |-- Validation
  +-- Error / Retry Queue
  |
  v
Result Import API
  |
  v
Result Store -> Verification -> Report
```

14.1 Connector requirements

- Connector runs close to the analyzer, typically on a Windows PC/local server.
- No analyzer should need to expose inbound ports directly to the public internet.
- Each analyzer has a configurable protocol and mapping profile.
- Raw messages are retained for troubleshooting according to retention policy.
- Patient/sample matching must be deterministic; uncertain matches go to an exception queue instead of silently attaching results.
- Imported results are distinguishable from manually entered results in audit history.
- Connection status, last message, last success, queue depth and error count are visible to authorized users.

15. Offline-First and Hybrid Deployment

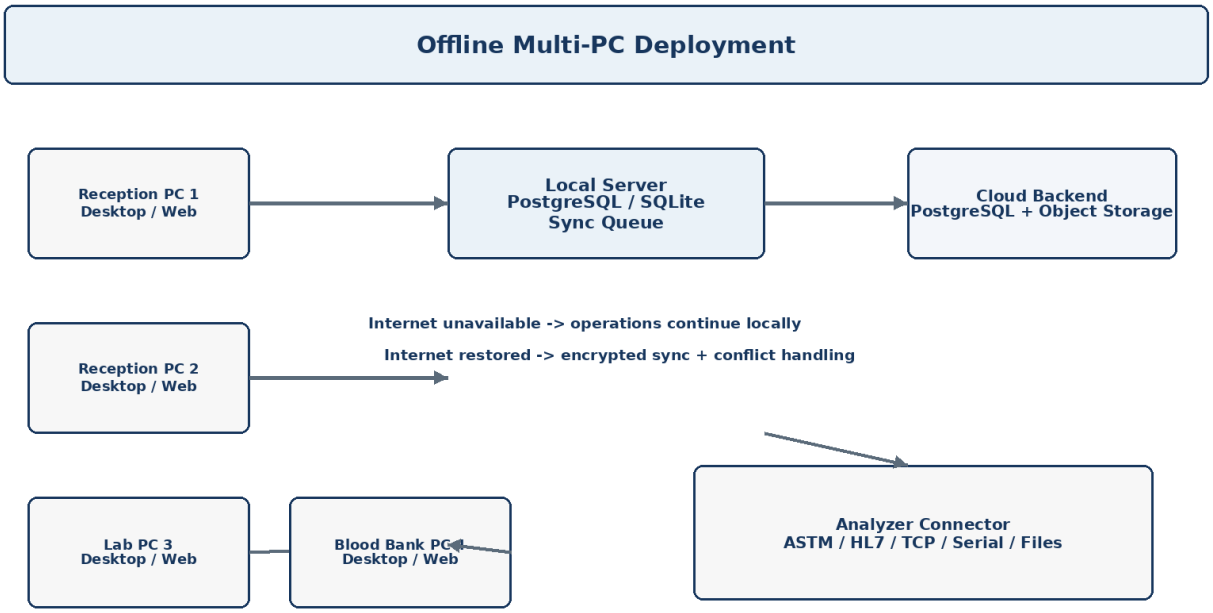


Figure 4. Offline multi-PC topology

For an offline diagnostic centre with multiple computers, a shared local server is preferable to giving each PC an independent database. The local server becomes the authoritative local source for the site, while cloud synchronization occurs when connectivity is restored.

15.1 Offline operating modes

Mode	Description
Cloud	All clients connect to central cloud services; recommended for normal SaaS deployments.
Local / Offline	Local server and local database provide continuity without internet.
Hybrid	Local operations continue during outages, then sync selected changes to cloud.
On-prem	Organization runs application services in its own environment; licensing and update policy remain managed by vendor.
Desktop standalone	Single-PC demo or very small installation; limited collaboration compared with local-server deployment.

15.2 Sync data model

Every syncable record should contain at minimum:

```
id
tenant_id
branch_id
created_at
updated_at
deleted_at
version
origin_device_id
sync_status
server_version

Change Queue:
local_change -> pending -> sent -> acknowledged -> applied
        -> conflict -> resolved
```

15.3 Conflict policy

- Never silently overwrite clinical results or blood-bank inventory with last-write-wins.
- Use server-side transactional checks for stock, payment and result approval.
- Patient demographics may use field-level merge in low-risk fields, but merge decisions should be logged.
- For clinical result conflicts, stop synchronization of the conflicting record and present a controlled resolution workflow to an authorized user.
- All conflict decisions create audit events.

16. SaaS Subscription and Offline License System

The commercial model separates subscription/entitlement data from clinical operations. A customer can purchase a cloud plan, an offline annual license or a hybrid plan with selected modules and limits.

Entity	Key fields
Plan	plan_code, name, billing_cycle, included_modules, user_limit, branch_limit, storage_limit
Subscription	tenant_id, plan_id, start_at, end_at, status, payment_reference, auto_renew
License	license_key, tenant_id, deployment_mode, issued_at, expires_at, features, limits, status
Device	device_id, tenant_id, branch_id, fingerprint, last_seen_at, license_status
Entitlement	tenant_id, module_code, enabled_from, enabled_to, limits
Renewal	previous_license, new_license, extension_days, payment_reference, actor

Expiry behavior

Do not delete data at expiry. The product should enter a controlled state such as renewal-required or restricted operational mode defined by contract. Read-only access and export/backup options should be planned so records remain recoverable. Exact grace-period and lock rules should be a commercial/legal decision.

17. Storage, Backup, Import and Export

17.1 File storage

- Object storage should hold reports, logos, signatures and attachments; database stores metadata and secure object references.
- Storage adapter interface should support cloud object storage and local filesystem storage.
- Files should have tenant isolation, checksums, size limits and retention metadata.
- Access should use short-lived signed URLs or server-mediated streaming rather than exposing permanent public file URLs.

17.2 Import/export

Data set	Formats	Rules
Patients	CSV/XLSX/JSON	Preview, field mapping, duplicate detection, validation
Tests	CSV/XLSX/JSON	Codes unique, units/ranges validated
Rates	CSV/XLSX	Effective dates and duplicate rule checks
Doctors/partners	CSV/XLSX	Partner type and identifiers required
Opening stock	CSV/XLSX	Batch/lot/expiry/quantity and unit rules
Reports	PDF/ZIP	Metadata linked to patient/report IDs where import is supported
Backup	Encrypted archive	Versioned backup and restore validation

18. Integration Strategy

Integration	Approach
ABDM / ABHA	Adapter layer; use current official implementation guides, consent flows and credentials. Store external identifiers

Integration	Approach
	separately from internal patient ID.
HL7 / FHIR	Interoperability gateway with canonical internal model. Map Patient, ServiceRequest/DiagnosticReport/Observation and related resources as applicable.
Analyzer	Local connector using device protocol adapters.
WhatsApp/SMS/Email	Notification provider adapters with templates and delivery status.
Payment gateway	Billing adapter; payment callbacks recorded as immutable payment events.
Storage	S3-compatible/local storage adapter.
External HIMS/EMR	API/webhook/HL7/FHIR connectors with tenant-specific configuration.

19. Technical System Architecture

19.1 Recommended technology stack

Layer	Technology	Why
Web	Nuxt 4 + Vue 3 + TypeScript	Fast unified app shell, strong component ecosystem, SSR/SPA flexibility
UI	Tailwind CSS + headless component primitives	Consistent design system and small reusable UI units
State	Pinia + query/cache layer	Predictable client state and server data caching
Backend	FastAPI + Python	Strong API productivity, validation, async support and integration libraries
Database	PostgreSQL 16+ target	Transactions, JSONB where needed, indexing, mature tooling
Cache / queue	Redis	Cache, rate limiting, ephemeral state and background job queues
Background jobs	Celery/ARQ or equivalent	Notifications, report generation, imports, sync and other asynchronous work
Desktop	Tauri + Vue	Lower footprint than Electron in many deployments; Windows-first packaging

Layer	Technology	Why
Mobile	Flutter	Single codebase for Android/iOS, suitable for scanning and field workflows
Local DB	SQLite for client cache/local operation; PostgreSQL for multi-PC local server	Simple single-PC storage plus stronger local multi-user server option
Reports	HTML/CSS renderer + PDF engine	One template model for preview and PDF
Object storage	S3-compatible abstraction	Cloud/local storage portability
Containers	Docker	Repeatable deployments, testing and local development
Reverse proxy	Nginx/Caddy/managed ingress	TLS termination and routing
Monitoring	OpenTelemetry + Prometheus/Grafana + error tracking	Traceability across API, workers and connectors
CI/CD	GitHub Actions/GitLab CI	Tests, lint, builds, migrations and deployment automation

19.2 Service boundaries

```
API Gateway / Web App
    |
    +-- Identity & RBAC
    +-- Patient Service
    +-- Lab Service
    +-- Billing Service
    +-- Referral Service
    +-- Blood Bank Service
    +-- Report Service
    +-- Inventory Service
    +-- Notification Service
    +-- Integration Service
    +-- Sync Service
    +-- Admin / License Service
```

Initial implementation may be a modular monolith. Extract services only after load, ownership or deployment constraints justify it.

Architecture recommendation

Start as a modular monolith with clear domain boundaries and background workers. Do not start with 12 independent microservices. This keeps development and debugging manageable while preserving a path to service extraction later.

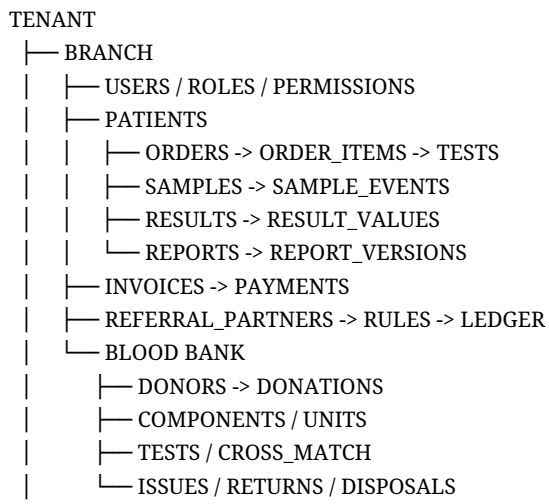
20. Database Design

The database must be relational-first. Clinical, billing and inventory workflows need transactions, constraints and consistent relationships. JSONB is useful for flexible configuration, report layout definitions and integration payloads, but core clinical and financial facts should remain strongly structured.

20.1 Database naming conventions

- Lowercase snake_case for table and column names.
- Singular conceptual names for major resources are acceptable; use one convention consistently.
- UUID/ULID identifiers for externally exposed records; numeric surrogate keys may be used internally if carefully isolated.
- Every tenant-owned table must include tenant_id unless the table is provably global.
- Dates use timestamptz in UTC at rest; display converts to organization timezone.
- Use created_at, updated_at; soft deletion only where business history requires it.
- Immutable clinical/financial events should be append-only where possible.

20.2 Core entity relationship map



20.3 Identity and tenancy tables

Table	Key columns / purpose
tenants	id, legal_name, display_name, status, timezone, country, settings_json
branches	id, tenant_id, name, code, address, timezone, status
departments	id, tenant_id, branch_id, name, type, status
users	id, tenant_id, username/email, password_hash, status, last_login_at
roles	id, tenant_id, name, description, is_system, status

Table	Key columns / purpose
permissions	id, code, module, resource, action, risk_level
role_permissions	role_id, permission_id, allow
user_roles	user_id, role_id, branch_scope_json, department_scope_json
user_permission_overrides	user_id, permission_id, allow, reason, expires_at
user_sessions	id, user_id, device_id, refresh_token_hash, issued_at, expires_at, revoked_at
devices	id, tenant_id, branch_id, device_type, fingerprint, app_version, last_seen_at
api_keys	id, tenant_id, name, key_hash, scopes, expires_at, status
audit_logs	id, tenant_id, actor_id, action, entity_type, entity_id, before_json, after_json, ip, created_at

20.4 Patient tables

Table	Key columns / purpose
patients	id, tenant_id, patient_no, first_name, last_name, dob, sex, mobile, email, status
patient_identifiers	id, patient_id, type, value, issuer, is_primary
patient_addresses	id, patient_id, type, address fields
patient_contacts	id, patient_id, contact_name, relationship, phone
patient_consents	id, patient_id, consent_type, status, captured_at, source
patient_documents	id, patient_id, document_type, object_key, checksum
patient_external_ids	id, patient_id, system, external_id, metadata_json
patient_merges	id, tenant_id, survivor_patient_id, merged_patient_id, reason, actor_id
patient_notes	id, patient_id, note_type, body, author_id

20.5 Test and configuration tables

Table	Key columns / purpose
lab_departments	id, tenant_id, code, name, status

Table	Key columns / purpose
specimen_types	id, tenant_id, code, name, handling
container_types	id, tenant_id, code, name, specimen_compatibility
tests	id, tenant_id, code, name, department_id, specimen_type_id, unit, method, status
test_parameters	id, test_id, code, name, datatype, unit, sort_order
test_reference_ranges	id, test_parameter_id, sex, age_min, age_max, low, high, critical_low, critical_high
test_calculations	id, test_id, formula, variables_json, version
test_profiles	id, tenant_id, code, name, department_id
profile_items	profile_id, test_id, sort_order
test_packages	id, tenant_id, code, name, price, status
package_items	package_id, test_id, quantity, sort_order
test_analyzer_mappings	id, analyzer_id, test_id, analyzer_code, parameter_code

20.6 Orders, samples and results tables

Table	Key columns / purpose
orders	id, tenant_id, branch_id, patient_id, order_no, status, referral_partner_id, ordered_at
order_items	id, order_id, test_id, profile_id/package_id, price, discount, tax, status
samples	id, order_id, patient_id, barcode, specimen_type_id, collected_at, received_at, status
sample_events	id, sample_id, event_type, event_at, actor_id, notes
worklists	id, tenant_id, department_id, worklist_date, status
worklist_items	worklist_id, sample_id, test_id, assigned_to, status
results	id, order_item_id, result_status, verification_status, entered_by, verified_by, approved_by
result_values	id, result_id, parameter_id, value_text, value_numeric, flag, unit, reference_snapshot_json
result_amendments	id, result_id, old_value_json, new_value_json, reason,

Table	Key columns / purpose
	actor_id
critical_notifications	id, result_id, threshold_type, recipient, notified_at, acknowledgment_status

20.7 Reporting tables

Table	Key columns / purpose
report_templates	id, tenant_id, code, name, report_type, status, current_version_id
report_template_versions	id, template_id, version_no, definition_json, schema_version, published_at, published_by
reports	id, tenant_id, patient_id, order_id, report_no, status, template_version_id, final_at
report_files	id, report_id, object_key, checksum, mime_type, generated_at
report_signatures	id, report_id, signer_user_id, signature_type, signed_at, signature_hash
report_delivery_events	id, report_id, channel, destination_hash, status, provider_message_id, sent_at
report_amendments	id, report_id, reason, previous_version_id, new_version_id, actor_id

20.8 Billing tables

Table	Key columns / purpose
rate_lists	id, tenant_id, branch_id, name, effective_from, effective_to, status
rate_list_items	rate_list_id, test_id/package_id, price
invoices	id, tenant_id, branch_id, patient_id, invoice_no, subtotal, discount, tax, total, status
invoice_items	id, invoice_id, order_item_id, item_type, gross, discount, tax, net, rule_snapshot_json
payments	id, invoice_id, payment_no, method, amount, status, provider_reference
refunds	id, payment_id, amount, reason, approved_by

Table	Key columns / purpose
cash_sessions	id, branch_id, user_id, opened_at, opening_cash, closed_at, closing_cash
cash_transactions	id, cash_session_id, invoice/payment/refund, amount, type
discount_rules	id, tenant_id, scope_json, discount_type, value, priority, effective dates
financial_adjustments	id, invoice_id, type, amount, reason, actor_id

20.9 Referral tables

Table	Key columns / purpose
referral_partners	id, tenant_id, partner_type, name, registration_no, status
referral_rules	id, partner_id, scope_json, discount_pct, commission_pct, fixed_commission, priority
referral_ledger	id, partner_id, invoice_id, amount, type, status, settlement_id
referral_settlements	id, partner_id, period_start, period_end, amount, status, approved_by
referral_contacts	id, partner_id, name, phone, email

20.10 Blood bank tables

Table	Key columns / purpose
donors	id, tenant_id, donor_no, name, dob, sex, blood_group, contact, status
donor_screenings	id, donor_id, questionnaire_version, eligibility, deferral_reason, screened_by
donor_deferrals	id, donor_id, start_at, end_at, reason, status
donations	id, donor_id, donation_no, collected_at, collection_site, status
blood_units	id, donation_id, unit_no, component_type_id, blood_group, volume, expiry_at, status
blood_components	id, code, name, storage_temp, shelf_life
blood_unit_tests	id, blood_unit_id, test_code, result, status, verified_by
blood_quarantine	id, blood_unit_id, reason, start_at, released_at

Table	Key columns / purpose
crossmatches	id, patient_id, sample_id, blood_unit_id, result, verified_by
blood_requests	id, tenant_id, patient_id, requester, urgency, status
blood_issues	id, blood_request_id, blood_unit_id, issued_at, issued_by
blood_returns	id, blood_issue_id, returned_at, reason, disposition
blood_disposals	id, blood_unit_id, disposed_at, reason, approved_by
adverse_events	id, patient_id, blood_issue_id, event_type, severity, reported_at

20.11 Inventory tables

Table	Key columns / purpose
inventory_items	id, tenant_id, item_type, code, name, unit, reorder_level
inventory_lots	id, item_id, lot_no, expiry_at, quantity, location_id
inventory_transactions	id, lot_id, transaction_type, quantity, source_ref, actor_id
inventory_locations	id, branch_id, name, type
inventory_alerts	id, lot_id/item_id, alert_type, status, created_at

20.12 Integration/sync tables

Table	Key columns / purpose
analyzers	id, tenant_id, branch_id, name, protocol, host, port, status
analyzer_connections	id, analyzer_id, connected_at, disconnected_at, status
analyzer_messages	id, analyzer_id, direction, message_type, raw_payload, received_at
analyzer_errors	id, analyzer_id, message_id, error_code, details, status
integration_endpoints	id, tenant_id, type, base_url, credential_ref, status
webhook_subscriptions	id, tenant_id, event_type, endpoint, secret_ref, status
sync_events	id, tenant_id, device_id, entity_type, entity_id, operation, payload_hash, status
sync_conflicts	id, sync_event_id, entity_type, entity_id, conflict_type,

Table	Key columns / purpose
	resolution, resolved_by
background_jobs	id, tenant_id, job_type, payload_json, status, attempts, run_at

20.13 Critical indexes and constraints

- Unique: (tenant_id, patient_no), (tenant_id, invoice_no), (tenant_id, order_no), (tenant_id, report_no).
- Index patient search on normalized mobile, patient_no, identifier values and name search strategy.
- Composite indexes for operational queues: (tenant_id, branch_id, status, updated_at).
- Blood stock indexes on (branch_id, blood_group, component_type_id, status, expiry_at).
- Result lookup indexes on order_item_id and verification/approval status.
- Use database constraints for non-negative financial amounts, valid quantity ranges and unique external identifiers.
- All foreign keys must have deliberate delete behavior. Do not cascade-delete clinical history by default.

21. API Design

```
/api/v1/auth/*
/api/v1/tenants/*
/api/v1/users/*
/api/v1/roles/*
/api/v1/patients/*
/api/v1/orders/*
/api/v1/samples/*
/api/v1/tests/*
/api/v1/results/*
/api/v1/reports/*
/api/v1/billing/*
/api/v1/referrals/*
/api/v1/blood-bank/*
/api/v1/inventory/*
/api/v1/analyzers/*
/api/v1/sync/*
/api/v1/imports/*
/api/v1/exports/*
/api/v1/admin/*
```

21.1 API rules

- All tenant-scoped APIs derive tenant_id from authenticated context, never from an untrusted frontend field alone.
- Authorization is evaluated server-side for every protected endpoint.
- Use idempotency keys for payment callbacks, sync replay and other operations that may be retried.
- Paginate all large collections; never return unbounded patient/result lists.
- Use cursor pagination for high-volume activity or transaction feeds.
- API error format should include code, message, field errors and correlation ID.
- Long-running exports, PDF batches and imports run as background jobs.

22. Security and Privacy Requirements

Area	Requirement
Authentication	Strong password policy, MFA option for administrators, secure session/refresh tokens, device/session revocation.
Authorization	Server-side RBAC; branch and department scope enforced in every query path.
Encryption	TLS in transit; encrypted storage and backups; secret manager for integration credentials.
Audit	Sensitive actions logged with actor, time, entity, reason and before/after metadata when appropriate.
Clinical changes	Result/report amendments require reason and preserve previous version.
File access	Private by default, signed/temporary access, tenant isolation.
Exports	Export permission is explicit and should be logged; bulk export may require elevated approval.
Payments	Do not store payment card credentials; use provider tokens/references.
Secrets	Never store analyzer passwords, API keys or signing credentials in source code.
Data retention	Configurable per tenant and applicable legal/clinical policy; retention changes audited.
Support access	Support impersonation should be time-limited, explicit, logged and preferably read-only by default.

23. Non-Functional Requirements

Category	Target / design goal
Page load	Common screens should feel interactive quickly; critical list pages should avoid blocking on large datasets.
API p95	Target under 500 ms for ordinary CRUD/query endpoints under expected load; heavier operations are asynchronous.
Concurrency	Support at least 50-100 simultaneous active users per tenant as initial target; architecture should scale beyond this.
Availability	Cloud target 99.9% service availability, excluding planned maintenance, subject to chosen infrastructure.

Category	Target / design goal
Offline	Core workflows available during network outage for supported deployments.
Data integrity	ACID transactions for invoices, payments, stock movement and result approval.
Scalability	Horizontal API/worker scaling; read replicas introduced when justified by load.
Observability	Logs, metrics, traces, error tracking, audit logs and connector health.
Accessibility	Keyboard-friendly forms, labels, contrast and responsive layouts.
Internationalization	English first; architecture ready for Hindi and other languages.

24. Backup, Disaster Recovery and Business Continuity

Control	Recommended policy
Database backup	Automated daily full + point-in-time recovery/WAL where infrastructure supports it.
Object storage	Versioning + backup/replication according to plan.
Offline backup	Encrypted local backup to selectable location.
Restore testing	Regular restore verification; backup is not considered valid until restore is tested.
RPO	Cloud target example: 15-60 minutes depending on plan and cost.
RTO	Cloud target example: 1-4 hours for standard plans.
Audit retention	Separate audit retention from operational record deletion policies.

25. QA and Validation Strategy

Layer	Tests
Unit	Pricing calculations, permission resolution, reference ranges, commission calculation, state transitions

Layer	Tests
API	CRUD, authorization, validation, tenancy isolation, idempotency
Integration	Analyzer parsing, message mapping, report renderer, notifications, payment callbacks
Database	Constraints, migrations, query performance, rollback behavior
UI	Registration, billing, result entry, report approval, responsive behavior
Offline	Create/update while offline, reconnect, sync order, conflict cases, duplicate prevention
Security	Tenant isolation, privilege escalation, session revocation, export restrictions
Performance	Concurrent reception usage, report batch generation, analyzer bursts, large patient searches
Recovery	Backup restore, corrupted queue, partial sync recovery
UAT	Real workflow validation with reception, technician, pathologist, blood-bank and accountant users

25.1 Must-pass scenarios

1. Four reception users register and bill different patients simultaneously without data mix-up.
2. Two users attempt to modify the same sensitive result; authorization and version checks prevent silent overwrite.
3. A sample is rejected with a reason and cannot be released as if it were accepted.
4. A report is amended after release and previous version remains accessible to authorized users.
5. A referral discount and commission are applied according to the active rule and preserved on the invoice.
6. Blood stock issue and return correctly update available quantity and audit history.
7. Internet is disconnected during active operations; supported offline workflows continue and later synchronize.
8. Analyzer sends a result; system maps it to the correct sample and test, or routes it to exception queue if matching is uncertain.
9. Tenant A cannot access Tenant B data using any direct identifier or API request.
10. Expired offline license does not erase local data and follows the configured renewal policy.

26. Implementation Roadmap

A realistic commercial implementation is better delivered in controlled releases. A reference plan below assumes a small-to-mid sized product team and existing engineering competence; actual dates should be re-estimated after detailed workflow workshops and analyzer/API validation.

Phase	Name	Deliverables	Reference duration
Phase 0	Discovery & architecture	Requirements workshops, UX flows, roles, deployment modes, regulatory gap analysis, domain model, prototype	2-3 weeks
Phase 1	Platform foundation	Tenancy, auth, RBAC, branches, users, audit, settings, design system, CI/CD	4-6 weeks
Phase 2	Pathology core	Patients, tests, orders, samples, barcodes, result entry, verification	6-8 weeks
Phase 3	Billing + referrals	Rates, invoices, payments, discount engine, referral commission ledger	4-6 weeks
Phase 4	Report engine	Template schema, renderer, designer, QR, signatures, starter templates	6-8 weeks
Phase 5	Analyzer integration	Connector service, protocol adapters, mapping, exception queues	4-8 weeks depending on devices
Phase 6	Blood bank	Donor, screening, components, stock, cross-match, issue/return/discard	8-12 weeks
Phase 7	Offline/hybrid	Local server, sync engine, conflict handling, offline license	6-10 weeks
Phase 8	Mobile	Staff/doctor/patient workflows, barcode/scan features	6-10 weeks
Phase 9	Hardening	Performance, security, backups, UAT, migration tools, release readiness	4-6 weeks

Release strategy

Release pathology + billing + reports first. Add blood bank after the core domain and audit model are proven. Offline and analyzer integration can run in parallel once the transactional APIs and data model stabilize.

27. Suggested Engineering Team

Role	Count	Primary responsibility
Product/Business Analyst	1	Workflow, requirements, acceptance criteria, stakeholder coordination
Tech Lead / Architect	1	Domain boundaries, data model, security, integrations, technical decisions
Backend Engineers	2-3	FastAPI, PostgreSQL, RBAC, billing, blood bank, sync
Frontend Engineer	2	Nuxt/Vue, design system, complex workflows
Desktop/Integration Engineer	1	Tauri, analyzer connector, local deployment, device communication
Mobile Engineer	1	Flutter staff/doctor/patient apps
QA Engineers	2	Manual + automation, device/integration testing, regression
DevOps/SRE	0.5-1	CI/CD, observability, backups, environments, security hardening
Domain SME(s)	Part-time	Laboratory, pathology, blood-bank and compliance workflow validation

28. Environments, DevOps and Release Management

```
Development
  |
Feature branch -> Pull Request -> Unit/API tests -> Lint/type checks
  |
Staging
  -> Migration test
  -> Integration tests
  -> UAT
  |
Production
  -> Migration backup
  -> Deploy
```

-> Smoke tests
-> Monitoring

- Use database migrations (Alembic) and never alter production schema manually except emergency procedures.
- Every production release is traceable to a git commit and migration version.
- Feature flags allow modules/features to be rolled out selectively.
- Desktop and analyzer connector releases should have explicit version compatibility with backend API versions.
- Keep staging data synthetic or de-identified; do not casually copy real patient data into development environments.

29. MVP Acceptance Criteria

Area	MVP must demonstrate
Unified UX	Single login and application shell exposes enabled modules.
Reception	At least four simultaneous users can register, bill, collect samples and print reports.
Patient	One patient identity connects order, sample, result, report and invoice.
RBAC	Create custom role and assign granular permissions; unauthorized actions are rejected server-side.
Pathology	Order -> sample -> result -> verification -> report workflow works end-to-end.
Billing	Rates, discount, payment, receipt, refund and audit trail work.
Referral	Doctor/hospital/lab referral can automatically apply configured discount and commission.
Reports	At least starter templates plus template versioning and QR verification work.
Analyzer	At least one documented analyzer protocol is integrated end-to-end in a controlled environment.
Offline	Local deployment works during internet outage and synchronizes after reconnection.
Security	Cross-tenant access tests fail closed; logs are generated for sensitive actions.
Backup	A backup can be restored to a test environment successfully.

30. Key Risks and Mitigations

Risk	Mitigation
Trying to build everything at once	Use modular releases and a strict MVP boundary.
Offline sync complexity	Start with local-server multi-PC mode before fully independent client databases; build a versioned change log early.
Analyzer variability	Use connector adapters and require a device test plan for each model/protocol.
Report template complexity	Define a stable template schema and renderer before building 100+ templates.
Regulatory assumptions	Keep compliance controls configurable and validate with current official guidance and the customer laboratory SOPs.
Performance regressions	Use indexes, pagination, async jobs and performance budgets from the beginning.
Permission sprawl	Use stable permission naming, risk levels, module groups and role templates.
Data corruption on sync	Transactions, idempotency, conflict queues and restore tooling; never silent last-write-wins for clinical/stock facts.
Data migration quality	Provide dry-run imports, error reports, mapping templates and reconciliation counts.

31. Future Expansion Roadmap

- Collection-centre network with sample dispatch/receipt tracking.
- Home sample collection and phlebotomist mobile workflow.
- Patient portal with report history and secure report verification.
- Advanced QC, lot/reagent management and equipment maintenance.
- AI-assisted document extraction or operational summarization, with human review and strict privacy controls.
- Enterprise data warehouse and cross-branch analytics.
- Additional interoperability implementation guides and national/regional health networks.
- PACS/radiology integration and broader HIMS integration as separate bounded modules.

32. Appendix A - Starter Permission Catalog

Module	Action	Example code
Patient	view	patient.view

Module	Action	Example code
Patient	create	patient.create
Patient	edit	patient.edit
Patient	merge	patient.merge
Patient	export	patient.export
Patient	documents.view	patient.documents.view
Patient	documents.manage	patient.documents.manage
Order	view	order.view
Order	create	order.create
Order	edit	order.edit
Order	cancel	order.cancel
Order	print	order.print
Sample	view	sample.view
Sample	collect	sample.collect
Sample	receive	sample.receive
Sample	reject	sample.reject
Sample	recollect	sample.recollect
Sample	print_barcode	sample.print_barcode
Result	view	result.view
Result	enter	result.enter
Result	edit	result.edit
Result	verify	result.verify
Result	approve	result.approve
Result	amend	result.amend
Result	export	result.export
Report	view	report.view
Report	generate	report.generate
Report	print	report.print

Module	Action	Example code
Report	download	report.download
Report	send	report.send
Report	amend	report.amend
Report	template.view	report.template.view
Report	template.manage	report.template.manage
Billing	view	billing.view
Billing	create	billing.create
Billing	edit	billing.edit
Billing	discount	billing.discount
Billing	payment	billing.payment
Billing	refund	billing.refund
Billing	cancel	billing.cancel
Billing	print	billing.print
Billing	export	billing.export
Billing	close_day	billing.close_day
Referral	partner.view	referral.partner.view
Referral	partner.manage	referral.partner.manage
Referral	rule.view	referral.rule.view
Referral	rule.manage	referral.rule.manage
Referral	ledger.view	referral.ledger.view
Referral	settlement.manage	referral.settlement.manage
BloodBank	donor.view	bloodbank.donor.view
BloodBank	donor.manage	bloodbank.donor.manage
BloodBank	screening.manage	bloodbank.screening.manage
BloodBank	donation.manage	bloodbank.donation.manage
BloodBank	stock.view	bloodbank.stock.view
BloodBank	stock.manage	bloodbank.stock.manage

Module	Action	Example code
BloodBank	crossmatch.manage	bloodbank.crossmatch.manage
BloodBank	issue	bloodbank.issue
BloodBank	return	bloodbank.return
BloodBank	discard	bloodbank.discard
BloodBank	adverse_event.manage	bloodbank.adverse_event.manage
Inventory	view	inventory.view
Inventory	create	inventory.create
Inventory	adjust	inventory.adjust
Inventory	issue	inventory.issue
Inventory	purchase	inventory.purchase
Inventory	expiry.manage	inventory.expiry.manage
Users/Roles	user.view	users/roles.user.view
Users/Roles	user.manage	users/roles.user.manage
Users/Roles	role.view	users/roles.role.view
Users/Roles	role.manage	users/roles.role.manage
Users/Roles	session.revoke	users/roles.session.revoke
Administration	tenant.manage	administration.tenant.manage
Administration	branch.manage	administration.branch.manage
Administration	settings.manage	administration.settings.manage
Administration	module.manage	administration.module.manage
Administration	license.manage	administration.license.manage
Administration	backup.manage	administration.backup.manage
Administration	audit.view	administration.audit.view
Administration	integration.manage	administration.integration.manage
Import/Export	import.execute	import/export.import.execute
Import/Export	export.execute	import/export.export.execute
Import/Export	export.bulk	import/export.export.bulk

Module	Action	Example code
Analytics	dashboard.view	analytics.dashboard.view
Analytics	finance.view	analytics.finance.view
Analytics	lab.view	analytics.lab.view
Analytics	bloodbank.view	analytics.bloodbank.view

33. Appendix B - Sample Role Matrix

Role	Patients	Samples/ Results	Reports	Billing	Blood Bank	Admin	Notes
Receptionist	P/C/E	P/C/E	View/Print	Create/ Payment	No	No	View
Billing Executive	View	View	View/Print	Full within limits	No	No	Finance
Lab Technician	View	Sample/Result	View	View	No	No	Technical
Pathologist	View	View	Verify/Approve	View	No	No	Clinical approval
Blood Bank Technician	View	View	View	View	Full	View stock	Blood bank
Blood Bank Manager	View	View	View	View	Full + approve	Full	Blood bank admin
Organization Admin	Full	Full	Full	Full	Configurable	Full	Tenant admin

34. Appendix C - Data Migration Plan

11. Inventory existing source data and identify authoritative masters.
12. Create mapping sheets for patients, tests, doctors, rates, opening balances and historical reports.
13. Normalize and deduplicate patients before import where possible.
14. Load master data first, then patients, then transactional history in dependency order.
15. Run dry-run import and generate an error file with row-level issues.
16. Reconcile counts and financial totals with source system.
17. Freeze source migration window only for final cutover; preserve source backup.
18. Run post-cutover verification and keep rollback plan until sign-off.

35. Appendix D - Example API Contracts

```
POST /api/v1/orders
Authorization: Bearer <token>
Idempotency-Key: <uuid>

{
  "patient_id": "PATIENT-ID",
  "branch_id": "BRANCH-ID",
  "items": [
    {"test_id": "CBC", "quantity": 1}
  ],
  "referral_partner_id": "DOC-ID"
}

Response:
{
  "order_id": "ORD-ID",
  "order_no": "ORD-2026-001025",
  "status": "ordered"
}
```

```
POST /api/v1/results/{result_id}/approve
{
  "reason": "Reviewed against analyzer output"
}

Server checks:
1. User permission
2. Result state
3. Version / optimistic lock
4. Required fields and flags
5. Audit event
6. Signature policy
7. Report release eligibility
```

36. Reference Sources and Standards to Validate During Implementation

The product should use current official documents and implementation guides during detailed design. The references below are starting points rather than a legal/compliance certification checklist.

Ref	Source	URL
R1	NABL FAQ - accreditation and ISO 15189 application/maintenance context	https://nabl-india.org/faq/
R2	NABL 112A - Specific Criteria for Accreditation of Medical Laboratories, Issue 01, 18-Dec-2024	https://nabl-india.org/nabl/file_download1.php?filename=202412180304-NABL-112-A-doc.pdf
R3	e-RaktKosh - centralized blood bank management overview	https://eraktkosh.mohfw.gov.in/BLDAHIMS/bloodbank/about.cnt

Ref	Source	URL
R4	e-RaktKosh portal - blood availability and blood centre services	https://eraktkosh.mohfw.gov.in/eraktkoshPortal/
R5	ABDM official material - ABHA, HFR/HPR and digital health ecosystem overview	https://abdm.gov.in/strapicms/uploads/ABDM_STANDEE_24aabea939.pdf
R6	HL7 FHIR specification directory	https://hl7.org/fhir/directory.html

Implementation note: healthcare interoperability and government integration requirements change. Before production, confirm the current ABDM implementation guides, credentials, consent flows, applicable blood-bank reporting requirements, current NABL criteria, and any organization-specific accreditation scope.

37. Final Product Definition

Target state

The finished product is a single configurable diagnostic platform: one login, one patient identity, one application shell, one billing engine, one report engine, shared RBAC, shared audit, shared inventory and optional blood-bank capabilities. It runs in cloud, local/offline and hybrid modes; supports web, Windows and mobile clients; connects to analyzers; and exposes controlled integration APIs.

The most important implementation decision is to keep clinical, financial, inventory and identity data as transactional domain models under one tenant boundary, while treating the desktop client, report designer, analyzer connector and mobile apps as clients/adapters over that model. This keeps the product configurable and maintainable as the number of tests, templates, branches, users and integrations grows.